

务, 例如文字处理、游戏。对于这类交互式任务来说, 它们需要在执行过程中响应用户操作, 因此周转时间并不是最重要的调度指标。这类任务的主要调度指标是响应时间 (Response Time), 即任务从被创建到第一次被处理器执行所需的时间。显然, 用户希望自己的请求 (例如键盘、鼠标输入) 能够被及时响应, 因此任务的响应时间越短越好。其计算公式是任务的第一次执行时间点与到达时间点之差:

$$T_{\text{响应}} = t_{\text{第一次执行}} - t_{\text{到达}}$$

顾名思义, 第一次执行时间点是任务第一次被 CPU 执行的时间点。显然, 该时间点越早, 则任务可以越快响应用户的请求。系统中所有任务的平均响应时间用于衡量任务响应用户是否及时, 同样是一个经典的调度指标。

实时性

操作系统还要处理有截止时间 (Deadline) 要求的实时任务。例如车载系统中的距离探测器会定时检测汽车与外部物体的距离, 如果当前车速过快且距离过近, 则车载系统会强制刹车。探测任务的实时性要求在限定时间内判断当前车速是否会导致碰撞, 从而避免严重的安全事故。又如视频的画面渲染, 每一帧画面需要在截止时间内完成进而保证视频的帧率, 否则会造成卡顿。在系统保证实时任务执行结果正确的同时, 调度还必须保证实时任务的实时性, 即任务在截止时间内完成。

资源利用率

在多核场景下存在多个 CPU 核心, 系统应该尽可能保证每个核心都在执行任务, 进而提高处理器资源的利用率。除了处理器资源外, 操作系统同时负责管理其他资源, 例如磁盘、网络、GPU 等。相对地, 任务也可能访问多种资源。如果调度器在调度任务时仅仅考虑处理器资源的使用情况, 虽然处理器资源会被充分利用, 但系统整体的资源利用率可能会很低, 系统性能无法达到最优。因此, 调度策略应尽可能保证系统资源被充分利用。

公平性

在通常情况下, 应保证每个任务都有被执行的可能。然而, 某些调度策略在倾向于调度一类任务时, 可能导致另一类任务一直无法被调度, 处于饥饿 (Starvation) 状态。上述情况对于无法被调度的任务是不公平的。因此, 调度策略应尽可能保证调度的公平性, 即每个任务都有机会被执行。

调度机制的开销

除了满足上述常见的调度指标外, 调度机制本身也会为系统整体带来开销。调度机制的开销可以分为两部分, 一部分是做出决策的开销, 另一部分则是任务上下文切换的开销。

调度器为了做出调度决策, 需要访问任务信息并进行计算与决策, 这一定会带来性能的开销。需要注意的是, 针对相同的调度策略, 不同调度器实现的计算复杂度可能会不同。假设系统中有若干任务, 一个调度器实现的复杂度为 $O(N)$, 即需要遍历所有任务才能做出决策。而另一个调度器实现的复杂度为 $O(1)$, 即常数时间就能做出决策。显然, 随着系统处理的并发任务数量的增长, 后者做出一次决策的开销会比前者少很多。

另一方面，处理器在任务间切换也会带来开销，当任务切换频繁时，其开销是无法忽略的。系统使用调度器的目的是优化任务的执行，而非引入新的性能瓶颈。因此，调度器应尽可能高效地实现，而调度策略也要控制任务调度的频率，避免调度机制的过大开销。

练习

7.2 根据所学的知识，请列举出操作系统中任务上下文切换可能导致的调度开销。

7.3 你是否还能想到其他调度指标？

在了解了上述调度指标后，不难发现许多指标之间存在冲突与权衡，这一现象也说明了选取合适的调度指标与策略绝非易事。例如，大部分情况下平均响应时间与调度机制开销是相互冲突的。如果系统希望减少任务的平均响应时间，就需要减少任务的时间片。但这反而导致处理器单位时间内在多个任务间进行频繁切换，放大了任务切换的开销。又如，大部分情况下实时性和公平性之间存在权衡。在系统中，某些实时任务为了在截止时间前完成，必须优先于其他任务执行，这可能会导致其他任务长时间无法执行，产生公平性问题。

7.3 经典调度策略

本节主要知识点

□ 有哪些经典的调度策略？

□ 不同的经典调度策略存在哪些优劣？

为了满足复杂多变的调度指标，多种多样的调度策略应运而生，在处理器调度发展的历史上留下了浓墨重彩的一笔。本章将通过经典调度策略（7.3节）、优先级调度策略（7.4节）和公平共享调度策略（7.5节），带领读者逐步构建起对处理器调度策略的理解。阅读完后续章节，相信读者可以具备初步的调度策略设计能力，满足一些简单的调度指标。

本节将从一些直观且经典的调度策略开始讲解，例如先到先得、最短任务优先、最短完成时间优先和时间片轮转等。了解这些经典的调度策略有助于理解更为复杂的调度策略背后的设计权衡，以及理解当下系统调度器的设计。

7.3.1 先到先得

在调度策略中，非常容易想到的一种策略是先到先得（First Come First Served, FCFS）策略。这一类策略也被称为先进先出（First In First Out, FIFO）策略。“先来后到”经常体现在日常生活中，在不同场合人们会自发地排队，比如买票、在银行办理业务等，都

体现了队列的思想。同样，FCFS 策略也是以队列为主要数据结构设计的。

该策略会在系统中维护一个运行队列，这个队列中的元素是处于就绪状态、等待执行的任务。在决定需要执行的任务时，FCFS 策略会选取运行队列中的第一个任务，将其移出运行队列并执行；当一个任务执行完后，它会被再次放入运行队列的队尾。运行队列起到为多个任务确定顺序的作用，保证任务的执行顺序与其进入运行队列的顺序一致。总体上，FCFS 策略最大的特点就是简单直观。开发者只需要维护一个队列就可以实现 FCFS 策略。同时，FCFS 策略不需要预知任务信息，也没有需要开发者手动调试的参数。

弊端 1：长短任务混合场景下对短任务不友好

我们通过一个例子来理解 FCFS 策略可能带来的问题。表 7.1 展示了某假设的任务场景一的任务信息，包括到达时间点和运行时间，该场景是一个长短任务混合场景。从第 0 秒起，一共有三个计算密集型任务 A、B、C 准备被处理，且它们不会发起 I/O 请求。任务 A、B、C 的到达时间点都为第 0 秒，假定它们进入队列的顺序为 A、B、C。图 7.2 展示了任务场景一基于 FCFS 策略的执行流，可以看到执行顺序同样是 A、B、C。表 7.2 中给出了任务的完成时间点和周转时间（周转时间 = 完成时间点 - 到达时间点）数据。首先看任务 A，它的周转时间与运行时间之比为 1 : 1，一般用户会根据任务的运行时间预估大致周转时间，因此 1 : 1 代表周转时间符合用户预期，是理想的调度情况。但是再看任务 B 和 C，它们的周转时间与运行时间之比分别高达 6 : 1 和 7 : 1。这是由于调度器首先调度任务 A，而任务 A 的运行时间较长，任务 B 和 C 不得不等待较长的时间才能开始运行。由于任务 B 和 C 的运行时间相对 A 较短，它们的周转时间会显著受到任务 A 的影响。可以想象，如果任务 A 的运行时间变得更长，那么这一问题会更加严重。这个场景引出了 FCFS 策略的第一个弊端：在短任务（短时运行任务）与长任务（长时运行任务）混合的应用场景下，FCFS 策略可能会导致短任务的周转时间与运行时间之比过大，对于短任务不友好。因此在长短任务混合场景下，FCFS 策略会导致用户体验较差。

表 7.1 任务场景一的任务信息

任务	到达时间点 (第 x 秒)	运行时间 (秒)
A	0	25
B	0	5
C	0	5

表 7.2 任务场景一在 FCFS 策略下的任务执行信息

任务	完成时间点 (第 x 秒)	周转时间 (秒)
A	25	25
B	30	30
C	35	35

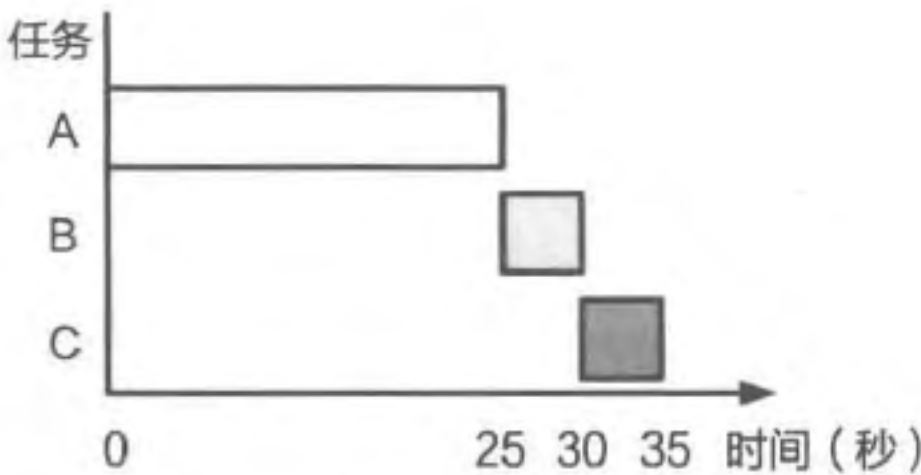


图 7.2 任务场景一在 FCFS 策略下的执行流

弊端 2：对 I/O 密集型任务不友好

I/O 密集型任务通常会花费大部分时间在等待 I/O，仅少量时间用于在 CPU 中处理请求。假设系统中有两个任务 A 和 B。任务 A 是一个计算密集型任务，它需要执行 40 秒。任务 B 是一个 I/O 密集型任务，先于任务 A 到达系统，它会循环两次以下步骤：①使用 CPU，时间 5 秒；②发起并等待一个耗时 15 秒的 I/O 请求。图 7.3 展示了这个例子基于 FCFS 策略的执行流。虽然 I/O 密集型任务 B 可以先执行，但是在其第一次等待 I/O 请求并变为阻塞状态后，计算密集型任务 A 会一直执行直到结束。因此，任务 B 即使在完成 I/O 请求之后也不能执行，只能延后发起下一轮的 I/O 请求，导致 I/O 资源的利用率降低。这个例子说明，FCFS 策略对于计算密集型任务更加友好，但可能会导致 I/O 密集型任务长时间内无法执行。

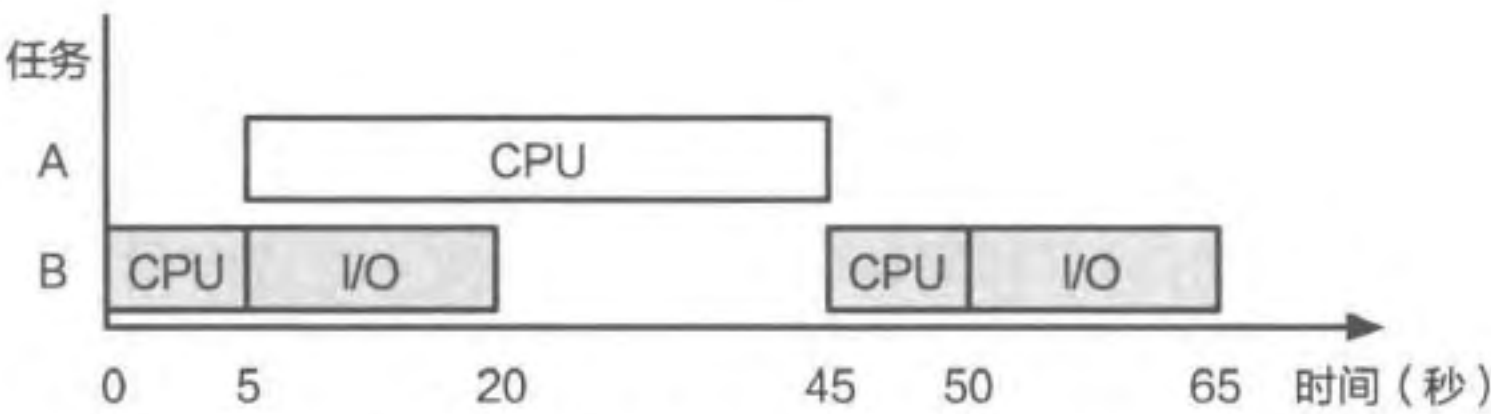


图 7.3 I/O 密集型任务场景在 FCFS 策略下的执行流

正如前文对于 FCFS 弊端的描述，我们意识到 FCFS 策略也因其简单直观的特性，而没有将许多重要因素纳入考量，导致它很难被直接应用在复杂系统中。不过，FCFS 仍然很适合作为系统实现早期的过渡调度策略，并且它的思想在后续介绍的调度策略中仍然得到了应用。

7.3.2 最短任务优先

FCFS 策略所面临的短任务等待时间过长的的问题促使我们寻找另一种调度策略，它应尽量让短任务立即执行。根据这一思想，得出的策略是最短任务优先（Shortest Job First, SJF）策略。顾名思义，该策略在调度时会选择运行时间最短的任务执行。

回顾表 7.1 中的任务场景一，表 7.3 给出了任务场景一在 SJF 策略下的任务执行信息，图 7.4 给出了对应的执行流。在 SJF 策略下，任务的平均周转时间相较于 FCFS 策略更短，那么是

表 7.3 任务场景一在 SJF 策略下的任务执行信息

任务	完成时间点（第 x 秒）	周转时间（秒）
A	35	35
B	5	5
C	10	10

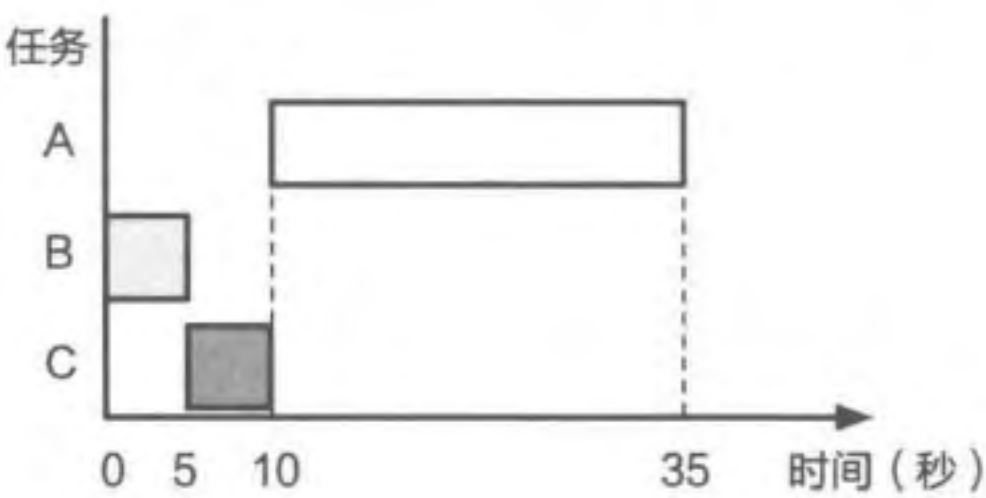


图 7.4 任务场景一在 SJF 策略下的执行流

否可以说 SJF 已经适应类似任务场景一的长短任务混合场景了呢？答案是否定的，SJF 策略仍然存在弊端。

弊端 1：必须预知任务运行时间

SJF 策略能够选取最短运行时间的前提是系统需要预知任务所需的运行时间。对于一些比较固定的应用场景，这个前提是合理的。但是在更多的场景下，系统很难预知将要处理的任务的确切运行时间，因为这取决于当前系统的状态和其他影响程序执行性能的因素。这一前提条件也限制了 SJF 的应用场景。

弊端 2：性能表现严重依赖任务到达时间点

回想表 7.1 中的任务场景一，仔细观察会发现三个任务的到达时间点都是第 0 秒。现在，我们对这个场景进行一定的修改。如表 7.4 的任务场景二所示，任务 A、B、C 的到达时间点分别被设为第 0 秒、第 1 秒和第 2 秒，运行时间不变。简单地说，任务场景二中，任务 B 和 C “迟到”了。图 7.5 展示了任务场景二在 SJF 策略下的执行流。虽然任务 B 和 C 仅仅晚到几秒，但是由于调度器在第 0 秒时并不知晓它们的存在，所以选择调度长任务 A。而当任务 B 和 C 到达时，调度器已经无法重新做出选择。现在的执行流同图 7.2 中的执行流一样，任务 B 和 C 仍必须等待任务 A 执行完。SJF 策略虽然看上去有效，但是它和 FCFS 策略一样，仍然过于简单，有许多因素需要进一步考虑。

表 7.4 任务场景二的任务信息

任务	到达时间点 (第 x 秒)	运行时间 (秒)
A	0	25
B	1	5
C	2	5

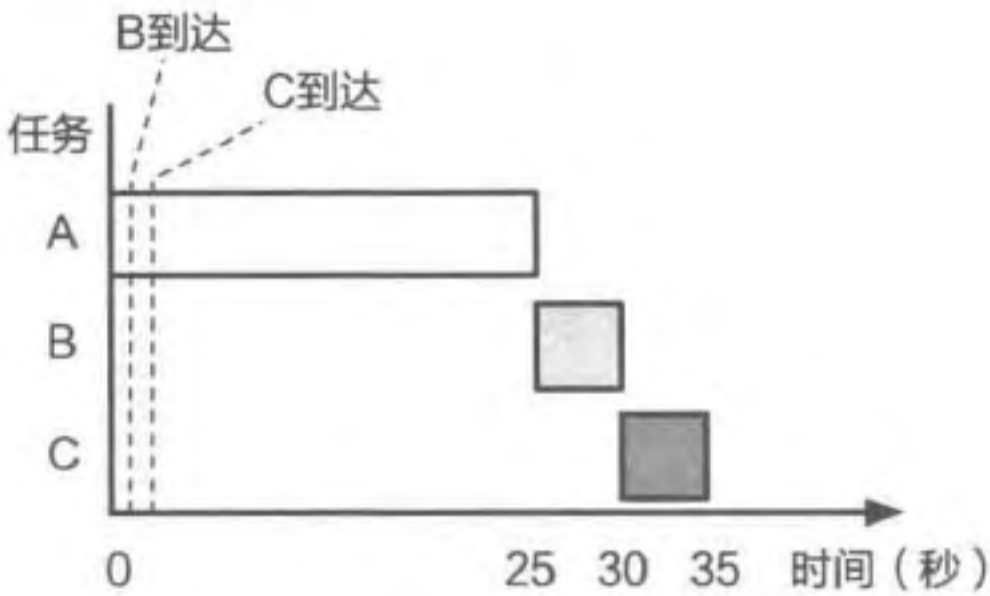


图 7.5 任务场景二在 SJF 策略下的执行流

总体上，SJF 策略对任务信息已知且要求短平均周转时间和大吞吐量的场景有比较好的效果。但由于其依赖任务到达时间点，所以适用场景相对受限。在下一节中我们会看到，SJF 策略也无法保证长任务的公平性。

7.3.3 最短完成时间优先

SJF 策略面临的问题是“迟到”的短任务无法从该策略中受益。为了解决这个问题，一个直接的思路是，是否可以让调度器在任务 B 和 C 到达时进行调度？答案是可以的，回顾 7.1.2 节介绍的对于调度时机的分类，我们可以利用抢占式的调度时机，让任务 B 和 C 无须等到任务 A 执行结束就能被调度。事实上，如果调度策略支持抢占式的调度时机，则可以被称为抢占式调度（Preemptive Scheduling）；否则，该调度策略被称为协作式调度（Non-Preemptive Scheduling）。前文介绍的 FCFS 策略和 SJF 策略，由于系统必

须等一个任务执行完或者主动退出执行才能调度，因此都是协作式调度。

接下来，根据上述思路，我们将介绍 SJF 策略的抢占式版本：最短完成时间任务优先（Shortest Time-to-Completion First, STCF）策略。再看表 7.4 所示的任务场景二，以及图 7.6 所示的该场景在 STCF 策略下的执行流，任务 B 和 C 的“迟到”并没有造成严重后果。由于任务 A 在第 0 秒首先到达系统，所以任务 A 执行了 1 秒。在第 1 秒时，任务 B 到达系统，由于其所需完成时间短于任务 A，所以任务 B 可以立即抢占任务 A 并执行。而任务 C 在第 2 秒到达时，任务 B 已经运行了 1 秒，任务 B 的完成时间点早于任务 C 的完成时间点，所以任务 C 不会抢占任务 B 的执行，而是等到任务 B 结束再开始执行。

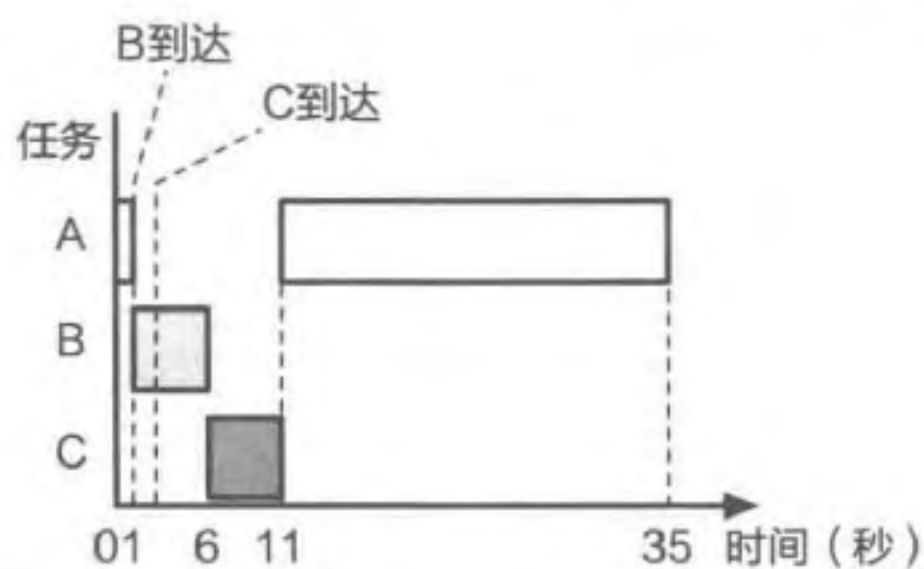


图 7.6 任务场景二在 STCF 策略下的执行流

弊端：长任务饥饿

STCF 策略极端倾向于完成时间较短的任务，因此当一个系统中有大量的短任务和少量的长任务时，这个系统的长任务很可能占用不到 CPU 资源，因而一直处于饥饿状态。在某些系统中，通常有后台服务（例如网络服务、数据库服务）随开机启动并一直执行，它们的运行时间相比于其他任务长很多。SJF 策略和 STCF 策略几乎不会让它们获得执行机会，而这对于整个系统调度的公平性是不利的。

总体上，STCF 策略扩展了 SJF 策略的适用场景，对于任务的平均周转时间和吞吐量有比较好的提升效果。但其仍然存在 SJF 策略的两个问题：需要预知任务执行时间，并且不保证长任务的公平性。

7.3.4 时间片轮转

之前讨论的策略主要是针对批处理任务的策略，它们主要关注的调度指标是周转时间。随着操作系统的发展，交互式程序（如终端）逐渐普及，与用户的交互为调度带来了新的指标——响应时间。响应时间指的是从用户发起请求到任务响应用户所需的时间。在生活中，相比于任务能否快速完成，用户可能更在意任务能否快速响应用户的请求。举一个简单的例子，一个程序需要一次性花 30 秒完成某项工作，但是在这个过程中没有任何提示；同样的工作，另一个程序可能要花费 40 秒完成，但是会每隔 1 秒告知用户当前的进度。在使用前者时，用户的耐心会随着时间的流逝逐渐耗尽，并最终倾向于认为这个程序不能响应。相对于前者的“冷淡”，在使用后者时，用户会有更好的体验。

前文讨论的策略只有在任务的到达和结束时才会进行调度，如果一段时间内没有新的任务到达或当前任务没有执行完，那么调度器就无法进行调度，因此非运行状态的任务也无法及时地响应用户。为了及时响应用户，需要为任务设置时间片，限定任务每次执行的时间。当前任务执行完时间片后，就切换到运行队列中的下一个任务。这一思想

的体现就是时间片轮转（Round Robin，RR）策略^[1]。图 7.7 展示了三个任务 A、B、C 在 RR 策略下的执行流。该 RR 策略使用 1 秒的时间片，以 3 秒为一个循环执行这三个任务。由于时间片一般会设为足够小，所以所有任务都可以在一定时间内执行并响应用户，从而将响应时间限定在一个可接受的范围内。相比于 SJF 策略和 STCF 策略，RR 策略不仅无须预知任务的运行时间，而且也不会出现长任务饥饿的情况。

对于 RR 策略，一个关键点是时间片大小该如何选取。理想情况下，时间片选取得越小，任务的响应时间就更短。但是，之前的讨论是基于抽象模型进行的，没有考虑调度器的调度开销和任务切换所导致的上下文切换开销。在实际场景中，过小的时间片反而会引入大量开销，使任务的调度成为严重的性能瓶颈。另一方面，如果时间片过长，比如 100 秒，RR 策略不仅无法满足用户对于响应时间的需求，还很可能产生与 FCFS 策略相同的负面效果。因此，如何将时间片降低到一个合理的值对于开发者是一大挑战，它要求开发者对整个应用场景有着明确的认知。

弊端：任务运行时间相似的场景下平均周转时间高

本节主要考虑的调度指标是响应时间，因此 RR 策略理应有着很好的表现。那么对于周转时间这一指标呢？再以图 7.7 的任务 A、B、C 为例，假设它们的运行时间都是 3 秒，那么它们会分别会在第 7 秒、第 8 秒、第 9 秒完成，这个执行流代表每个任务都在最后时刻完成，它们的平均周转时间非常高。RR 策略在一定程度上保证了每个任务之间的公平性（每个任务都能在一定的时间被调度），同时也获得良好的响应时间。但是，在特定场景（如每个任务运行时间相似）下，任务的平均周转时间可能较差。

总体上，RR 策略是一个非常通用的策略，在批处理任务和交互式任务混合场景下有相对较好的综合表现，且无须预知任务执行时间。但 RR 策略不适用于任务运行时间相似且对平均周转时间有极高要求的场景。

7.3.5 经典调度策略的比较

本节介绍了经典的 FCFS、SJF、STCF 和 RR 调度策略，对这些策略的总结如表 7.5 所示。

表 7.5 经典调度策略的总结

	优 势	弊 端
先到先得（FCFS）	设计实现简单	长短任务混合场景下对短任务不友好
	对所有任务公平	对 I/O 密集型任务不友好
最短任务优先（SJF）	对短任务友好	必须预知任务运行时间
		性能表现严重依赖任务到达时间点
		对长任务不公平

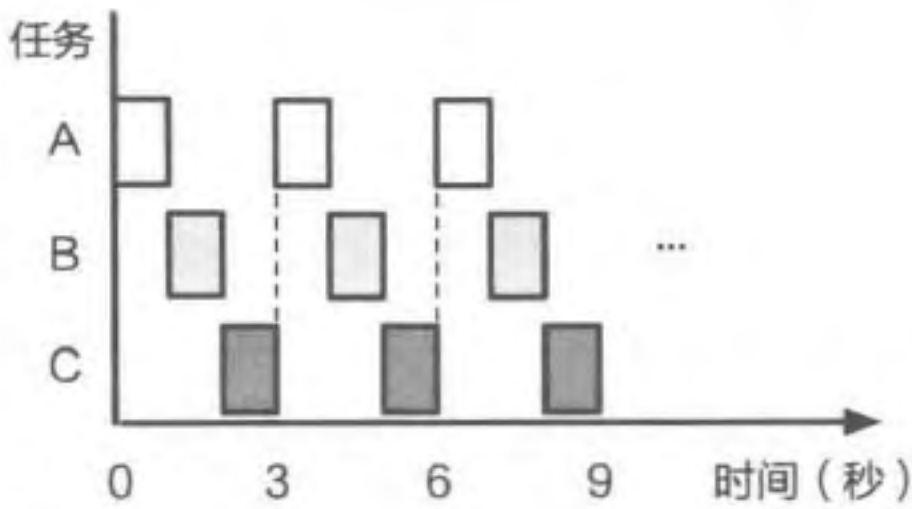


图 7.7 RR 策略在任务运行时间相似场景下的执行流

(续)

	优势	弊端
最短完成时间优先 (STCF)	对短任务友好	必须预知任务运行时间
	支持抢占	对长任务不公平
时间片轮转 (RR)	任务平均响应时间低	任务运行时间相似的场景下平均周转时间高
	对所有任务公平	

7.4 优先级调度策略

本节主要知识点

- ❑ 优先级与不同调度策略的关系是什么？
- ❑ 有哪些典型的优先级调度？
- ❑ 如何设置任务的优先级？

在了解了经典调度策略后，本节将介绍应用广泛的优先级调度。现假设以下场景：小明在一个单核计算机上通过多个后台程序处理大数据集，这些后台程序会在指定的文件目录中输出中间结果文件。小明希望通过命令行的 `ls` 指令看到数据处理的进度。我们假设系统使用 `RR` 策略进行调度，那么现在系统中有 N 个批处理任务和一个命令行的交互式任务。基于 `RR` 策略的调度不会对管理的任务进行区分，因此单位时间内命令行只会占用 $1/(N+1)$ 的 CPU 时间。大部分时间下，后台程序会阻塞 `ls` 指令的执行，小明会感觉整个系统很卡顿，就连 `ls` 都需要很长时间才能响应。如果小明脾气不好，他很有可能马上通过 `kill` 指令终止后台程序。因此，为了保持良好的用户体验，调度器应尽量避免交互式任务被批处理任务阻塞。

如果操作系统有能力让交互式任务优先于批处理任务执行，就能避免交互式任务被批处理任务阻塞。为此，调度器引入了优先级的概念。优先级是一个直接有效的概念，通过为每个任务指定优先级，调度器可以确认哪些任务应该优先执行。只要操作系统可以分辨一个任务是交互式任务还是批处理任务^①，就能为交互式任务设置更高的优先级，进而为用户提供更好的体验。

其实，之前讨论的算法都隐式地使用了优先级这一概念。表 7.6 给出了 7.3 节讨论的调度策略以及对应策略确定任务优先级的方式。可以看到，虽然我们讨

表 7.6 不同调度策略对应的优先级确定方式

调度策略	优先级确定方式
FCFS	任务等待时间长
SJF	任务运行时间短
STCF	任务剩余完成时间短
RR	所有任务平等

① 操作系统可以通过让程序员显式指定的方式或通过启发式算法自动判断的方式判断一个任务的种类。